

UNIT III

Dynamic Programming: General method – multi-stage graphs – all pair shortest path algorithm – 0/1 Knapsack and Traveling salesman problem – chained matrix multiplication. Basic Search and Traversal technique: Techniques for binary trees and graphs – AND/OR graphs – biconnected components – topological sorting.

2 Marks

1. Write the difference between the Greedy method and Dynamic programming. Greedy method Dynamic programming

1. Only one sequence of decision is generated.
1. Many number of decisions are generated.
2. It does not guarantee to give an optimal solution always.
2. It definitely gives an optimal solution always.

2. Define dynamic programming. (UQ APRIL'12/APRIL 2015)

Dynamic programming is an algorithm design method that can be used when a solution to the problem is viewed as the result of sequence of decisions.

3. What are the features of dynamic programming?

Optimal solutions to sub problems are retained so as to avoid re-computing their values. Decision sequences containing subsequences that are sub optimal are not considered. It definitely gives the optimal solution always.

4. What are the drawbacks of dynamic programming?

Time and space requirements are high, since storage is needed for all level. Optimality should be checked at all levels.

5. Write the general procedure of dynamic programming. (UQ APRIL'13)

The development of dynamic programming algorithm can be broken into a sequence of 4 steps.

1. Characterize the structure of an optimal solution.
2. Recursively defines the value of the optimal solution.
3. Compute the value of an optimal solution in the bottom-up fashion.
4. Construct an optimal solution from the computed information.

6. Define principle of optimality.

It states that an optimal sequence of decisions has the property that whenever the initial stage or decisions must constitute an optimal sequence with regard to stage resulting from the first decision.

7. Give an example of dynamic programming and explain.

An example of dynamic programming is knapsack problem. The solution to the knapsack problem can be viewed as a result of sequence of decisions. We have to decide the value of x_i for $1 \leq i \leq n$. First we make a decision on x_1 and then on x_2 and so on. An optimal sequence of decisions maximizes the object function

8. Explain any one method of finding the shortest path.

One way of finding a shortest path from vertex i to j in a directed graph G is to decide which vertex should be the second, which is the third, which is the fourth, and so on, until vertex j is reached. An optimal sequence of decisions is one that results in a path of least length.

9. Define 0/1 knapsack problem.

The solution to the knapsack problem can be viewed as a result of sequence of decisions. We have to decide the value of x_i . x_i is restricted to have the value 0 or 1 and by using the function $\text{knap}(l, j, y)$ we can represent the problem as maximum $\sum p_i x_i$ subject to $\sum w_i x_i \leq y$ where l - iteration, j - number of objects, y - capacity.

10. What is the formula to calculate optimal solution in 0/1 knapsack problem?

The formula to calculate optimal solution is $g_0(m) = \max\{g_1, g_1(m-w_1)+p_1\}$.

11. Write about traveling salesperson problem.

Let $G = (V, E)$ be a directed. The tour of G is a directed simple cycle that includes every vertex in V . The cost of a tour is the sum of the cost of the edges on the tour. The traveling salesperson problem to find a tour of minimum cost.

12. Write some applications of traveling salesperson problem.

Routing a postal van to pick up mail from boxes located at n different sites.

- Using a robot arm to tighten the nuts on some piece of machinery on an assembly line. Production environment in which several commodities are manufactured on the same set of machines.

13. Give the time complexity and space complexity of traveling salesperson problem.

Time complexity is $O(n^2 2^n)$.

Space complexity is $O(n 2^n)$.

14. List different traversal technique in graph (UQ APRIL'12/ APRIL 2015)

1. DFS and
2. BFS.

15. What is biconnected component? (UQ Nov'12)

Graph with no articulation point then it is called as biconnected component

16. What is articulation point?

When a node in a graph is deleted, the graph divided into two or more graph then the node is called articulation point.

17. Write about multistage graph with example (UQ APRIL'13 & Nov'12, APRIL/MAY'14)

A multistage graph $G = (V, E)$ is a directed graph in which the vertices are portioned into $K \geq 2$ disjoint sets V_i , $1 \leq i \leq k$. In addition, if $\langle u, v \rangle$ is an edge in E , then $u \in V_i$ and $v \in V_{i+1}$ for some i , $1 \leq i < k$.

18. What is all pair shortest path problem?(APRIL/MAY'14)

Let $G(V, E)$ be a directed graph with n vertices, ' E ' is the set of edges & V is the set of n vertices. Each edge has an associated non-negative length. Calculate the length of the shortest path between each pair of nodes i.e Shortest path between every vertex to all other vertices. The all pairs shortest path problem is to determine a matrix A such that $A(i, j)$ is the length of a shortest path from i to j .

The principle of optimality: If k is the node on the shortest path from i to j then the part of the path from i to k and the part from k to j must also be optimal, that is shorter.

1. Explain dynamic program general method

- It is an algorithm design method that can be used when the solution to a problem can be viewed as the result of a sequence of decisions.
- The idea of dynamic programming is thus quit simple: avoid calculating the same thing twice, usually by keeping a table of known result that fills up a sub instances are solved.
- Divide and conquer is a top-down method. When a problem is solved by divide and conquer, we immediately attack the complete instance, which we then divide into smaller and smaller sub-instances as the algorithm progresses.
- Dynamic programming on the other hand is a bottom-up technique. We usually start with the smallest and hence the simplest sub- instances. By combining their solutions, we obtain the answers to sub-instances of increasing size, until finally we arrive at the solution of the original instances.
- The essential difference between the greedy method and dynamic programming is that the greedy method only one decision sequence is ever generated. In dynamic programming, many decision sequences may be generated. However, sequences containing sub-optimal sub-sequences cannot be optimal and so will not be generated.
- Because of principle of optimality, decision sequences containing subsequences that are suboptimal are not considered. Although the total number of different decision sequences is exponential in the number of decisions(if there are d choices for each of the n decisions to be made then there are d^n possible decision sequences),Dynamic programming algorithms often have a polynomial complexity.

2. Explain multistage graph with example (UQ :APRIL/MAY'14)

1. A multistage graph $G = (V, E)$ is a directed graph in which the vertices are portioned into $K \geq 2$ disjoint sets $V_i, 1 \leq i \leq k$.
2. In addition, if $\langle u, v \rangle$ is an edge in E , then $u \in V_i$ and $v \in V_{i+1}$ for some $i, 1 \leq i < k$.
3. If there will be only one vertex, then the sets V_i and V_k are such that $|V_i| = |V_k| = 1$.
4. Let 's' and 't' be the source and destination respectively.
5. The cost of a path from source (s) to destination (t) is the sum of the costs of the edger on the path.
6. The *MULTISTAGE GRAPH* problem is to find a minimum cost path from 's' to 't'.
7. Each set V_i defines a stage in the graph. Every path from 's' to 't' starts in stage-1, goes to stage-2 then to stage-3, then to stage-4, and so on, and terminates in stage-k.
8. This *MULTISTAGE GRAPH* problem can be solved in 2 ways.
 - a) Forward Method.
 - b) Backward Method.

FORWARD METHOD

Assume that there are 'k' stages in a graph. In this *FORWARD* approach, we will find out the cost of each and every node starting from the 'k' th stage to the 1st stage. We will find out the path (i.e.) minimum cost path from source to the destination (i.e.) [Stage-1 to Stage-k].

PROCEDURE:

- Maintain a cost matrix cost (n) which stores the distance from any vertex to the destination.
- If a vertex is having more than one path, then we have to choose the minimum distance path and the intermediate vertex which gives the minimum distance path will be stored in the distance array 'D'.
- In this way we will find out the minimum cost path from each and every vertex.
- Finally cost (1) will give the shortest distance from source to destination.

- For finding the path, start from vertex-1 then the distance array $D(1)$ will give the minimum cost neighbor vertex which in turn give the next nearest vertex and proceed in this way till we reach the Destination.
- For a 'k' stage graph, there will be 'k' vertex in the path.
- In the above graph $V1 \dots V5$ represent the stages. This 5 stage graph can be solved by using forward approach as follows,

ALGORITHM:

Algorithm FGraph (G,k,n,p)

// The i/p is a k-stage graph $G=(V,E)$ with 'n' vertex indexed in order of stages.

// E is a set of edges.

// $c[i,j]$ is the cost of $\langle i,j \rangle$

// $p[1:k]$ is a minimum cost path.

{

cost[n]=0.0;

for $j=n-1$ to 1 step-1 do

{

//compute cost[j],

Let 'r' be the vertex such that $\langle j,r \rangle$ is an edge of 'G' &

$c[j,r] + \text{cost}[r]$ is minimum.

$\text{cost}[j] = c[j,r] + \text{cost}[r];$

$d[j] = r;$

}

// find a minimum cost path.

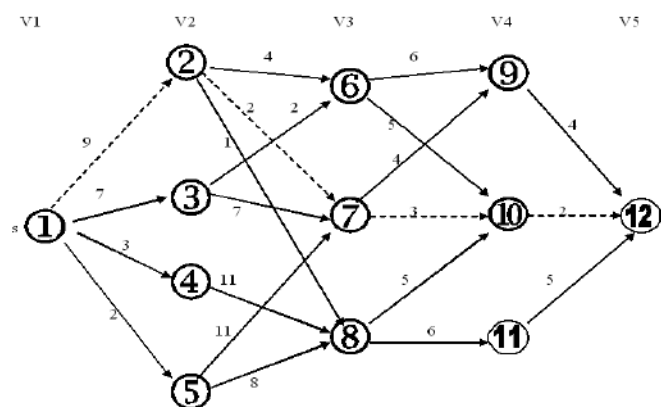
$p[1]=1;$

$p[k]=n;$

for $j=2$ to $k-1$ do

$p[j]=d[p[j-1]];$

}



TRACE OUT:

n=12 k=5
cost (12)=0 d (12)=0
for j = 11 to 1
When j=11
 cost (11)=5 d (11)=12
When j=10
 cost (10)=2 d (10)=12
When j=9
 cost (9)=4 d (9)=12
When j=8

Note:

If there is more than one path from vertex j to next stage vertices, then we find the closest vertex by the following formula:

- For forward approach,

$$\begin{aligned} \text{Cost (i,j)} &= \min \{ C (j,l) + \text{Cost (i+1,l)} \} \\ l &\in V_{i+1} \\ (j,l) &\in E \end{aligned}$$

$$\begin{aligned} \text{cost}(8) &= \min \{ C (8,10) + \text{Cost (10)}, C (8,11) + \text{Cost (11)} \} \\ &= \min (5 + 2, 6 + 5) \\ &= \min (7, 11) \\ &= 7 \end{aligned}$$

$$\text{cost}(8) = 7 \quad d(8)=10$$

When j=7

$$\begin{aligned} \text{cost}(7) &= \min (c (7,9) + \text{cost}(9), c (7,10) + \text{cost}(10)) \\ &= \min (4+4, 3+2) \\ &= \min (8, 5) \\ &= 5 \end{aligned}$$

$$\text{cost}(7) = 5 \quad d(7) = 10$$

When j=6

$$\begin{aligned} \text{cost}(6) &= \min (c (6,9) + \text{cost}(9), c (6,10) + \text{cost}(10)) \\ &= \min (6+4, 5 + 2) \\ &= \min (10, 7) \\ &= 7 \end{aligned}$$

$$\text{cost}(6) = 7 \quad d(6) = 10$$

When j=5

$$\begin{aligned} \text{cost}(5) &= \min (c (5,7) + \text{cost}(7), c (5,8) + \text{cost}(8)) \\ &= \min (11+5, 8 + 7) \\ &= \min (16, 15) \\ &= 15 \end{aligned}$$

$$\text{cost}(5) = 15 \quad d(5) = 18$$

When j=4

$$\begin{aligned} \text{cost}(4) &= \min (c (4,8) + \text{cost}(8)) \\ &= \min (11+7) \\ &= 18 \end{aligned}$$

$$\text{cost}(4) = 18 \quad d(4) = 8$$

When $j=3$

$$\begin{aligned} \text{cost}(3) &= \min(c(3,6) + \text{cost}(6), c(3,7) + \text{cost}(7)) \\ &= \min(2+7, 7+5) \\ &= \min(9, 12) \\ &= 9 \end{aligned}$$

$$\text{cost}(3) = 9 \quad d(3) = 6$$

When $j=2$

$$\begin{aligned} \text{cost}(2) &= \min(c(2,6) + \text{cost}(6), c(2,7) + \text{cost}(7), c(2,8) + \text{cost}(8)) \\ &= \min(4+7, 2+5, 1+7) \\ &= \min(11, 7, 8) \\ &= 7 \end{aligned}$$

$$\text{cost}(2) = 7 \quad d(2) = 7$$

When $j=1$

$$\begin{aligned} \text{cost}(1) &= \min(c(1,2) + \text{cost}(2), c(1,3) + \text{cost}(3), c(1,4) + \text{cost}(4), c(1,5) + \text{cost}(5)) \\ &= \min(9+7, 7+9, 3+18, 2+15) \\ &= \min(16, 16, 21, 17) \\ &= 16 \end{aligned}$$

$$\text{cost}(1) = 16 \quad d(1) = 2$$

$$p[1]=1 \quad p[5]=12$$

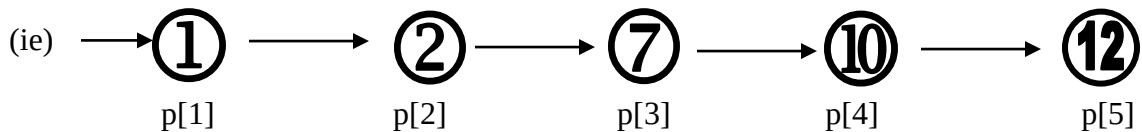
for $j=2$ to 4

$$\text{When } j=2 \quad p[2]=d[p[1]]=d[1]=2$$

$$\text{When } j=3 \quad p[3]=d[p[2]]=d[2]=7$$

$$\text{When } j=4 \quad p[4]=d[p[3]]=d[7]=10$$

The path through which you have to find the shortest distance from vertex 1 to vertex 12.



Start from vertex - 1

$$D(1) = 2$$

$$D(2) = 7$$

$$D(7) = 10$$

$$D(10) = 12$$

So, the minimum –cost path is,



$\therefore$ The cost is $9+2+3+2=16$

ANALYSIS:

The time complexity of this forward method is $O(V + E)$

BACKWARD METHOD

- If there one 'K' stages in a graph using back ward approach. we will find out the cost of each & every vertex starting from 1st stage to the kth stage.
- We will find out the minimum cost path from destination to source (i.e.) [from stage k to stage 1]

PROCEDURE:

1. It is similar to forward approach, but differs only in two or three ways.
2. Maintain a cost matrix to store the cost of every vertices and a distance matrix to store the minimum distance vertex.
3. Find out the cost of each and every vertex starting from vertex 1 up to vertex k.
4. To find out the path star from vertex 'k', then the distance array D (k) will give the minimum cost neighbor vertex which in turn gives the next nearest neighbor vertex and proceed till we reach the destination.

ALGORITHM :

Algorithm BGraph (G,k,n,p)

// The i/p is a k-stage graph $G=(V,E)$ with 'n' vertex indexed in order of stages

// E is a set of edges.

// $c[i,j]$ is the cost of $\langle i,j \rangle$

// $p[1:k]$ is a minimum cost path.

```
{
    bcost[1]=0.0;
    for j=2 to n do
    {
        //compute bcost[j],
        Let 'r' be the vertex such that  $\langle r,j \rangle$  is an edge of 'G' &  $bcost[r]+c[r,j]$  is
        minimum.
        bcost[j] = bcost[r] + c[r,j];
        d[j] =r;
    }
    // find a minimum cost path.
    p[1]=1;
    p[k]=n;
    for j= k-1 to 2 do
        p[j]=d[p[j+1]];
}
```

TRACE OUT:

n=12	k=5	
bcost (1)=0		d (1)=0
for j = 2 to 12		
When j=2		
bcost(2) = 9		d(2)=1
When j=3		

$\text{bcost}(3) = 7$ $d(3)=1$
 When $j=4$
 $\text{bcost}(4) = 3$ $d(4)=1$
 When $j=5$
 $\text{bcost}(5) = 2$ $d(5)=1$
 When $j=6$
 $\text{bcost}(6) = \min(c(2,6) + \text{bcost}(2), c(3,6) + \text{bcost}(3))$
 $\quad = \min(13, 9)$
 $\text{bcost}(6) = 9$ $d(6)=3$
 When $j=7$
 $\text{bcost}(7) = \min(c(3,7) + \text{bcost}(3), c(5,7) + \text{bcost}(5), c(2,7) + \text{bcost}(2))$
 $\quad = \min(14, 13, 11)$
 $\text{bcost}(7) = 11$ $d(7)=2$

 When $j=8$
 $\text{bcost}(8) = \min(c(2,8) + \text{bcost}(2), c(4,8) + \text{bcost}(4), c(5,8) + \text{bcost}(5))$
 $\quad = \min(10, 14, 10)$
 $\text{bcost}(8) = 10$ $d(8)=2$
 When $j=9$
 $\text{bcost}(9) = \min(c(6,9) + \text{bcost}(6), c(7,9) + \text{bcost}(7))$
 $\quad = \min(15, 15)$
 $\text{bcost}(9) = 15$ $d(9)=6$
 When $j=10$
 $\text{bcost}(10) = \min(c(6,10) + \text{bcost}(6), c(7,10) + \text{bcost}(7), c(8,10) + \text{bcost}(8))$
 $\quad = \min(14, 14, 15)$
 $\text{bcost}(10) = 14$ $d(10)=6$
 When $j=11$
 $\text{bcost}(11) = c(8,11) + \text{bcost}(8)$
 $\text{bcost}(11) = 16$ $d(11)=8$
 When $j=12$
 $\text{bcost}(12) = \min(c(9,12) + \text{bcost}(9), c(10,12) + \text{bcost}(10), c(11,12) + \text{bcost}(11))$
 $\quad = \min(19, 16, 21)$
 $\text{bcost}(12) = 16$ $d(12)=10$

$p[1]=1$ $p[5]=12$
 for $j=4$ to 2
 When $j=4$ $p[4]=d[p[5]]=d[12]=10$
 When $j=3$ $p[3]=d[p[4]]=d[10]=6$
 When $j=2$ $p[2]=d[p[3]]=d[6]=3$

The path through which you have to find the shortest distance from vertex 1 to vertex 12.

$p[5]=12$
 $p[4]=10$
 $p[3]=6$
 $p[2]=3$
 $p[1]=1$

So the minimum cost path is,

$1 \xrightarrow{7} 3 \xrightarrow{2} 6 \xrightarrow{5} 10 \xrightarrow{2} 12$

The cost is 16.

3. Explain all pair shortest path

- Let $G(V,E)$ be a directed graph with n vertices, 'E' is the set of edges & V is the set of n vertices.
- Each edge has an associated non-negative length.
- We want to calculate the length of the shortest path between each pair of nodes.
- i.e) Shortest path between every vertex to all other vertices.
- Suppose the nodes of G are numbered from 1 to n , so $n=\{1,2,\dots,n\}$
- $\text{cost}(i,j)$ is length of edge $\langle i,j \rangle$ and it is called as cost adjacency matrix.
 - $\text{cost}(i,i)=0$ for $i=1,2,\dots,n$,
 - $\text{cost}(i,j)=\text{cost of edge } \langle i,j \rangle$ for all $i \neq j$
 - $\text{cost}(i,j)=\text{infinity}$, if the edge (i,j) does not exist.
- The all pairs shortest path problem is to determine a matrix A such that $A(i,j)$ is the length of a shortest path from i to j .
- The principle of optimality:
 - If k is the node on the shortest path from i to j then the part of the path from i to k and the part from k to j must also be optimal, that is shorter.
- First create a cost adjacency matrix for the given graph.
- Copy the above matrix to matrix D , which will give the direct distance between nodes.
- We have to perform $n*n$ iterations for each iteration of k . The matrix D will give you the distance between nodes with only $(1,2,\dots,k)$ as intermediate nodes.
- At the iteration k , we have to check for each pair of nodes (i,j) whether or not there exists a path from i to j passing through node k .
- Likewise we have to find the value for n iterations (ie) for n nodes.

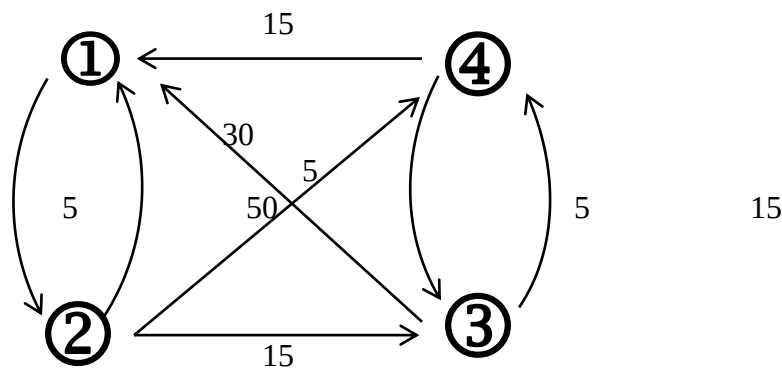


Fig: floyd's algorithm and work

ALGORITHM :

Algorithm Allpaths(cost,A,n)

//cost[1...n,1...n] is the cost adjacency matrix of graph with n vertices

//A[i,j] is the cost of shortest path from vertex i to j

//cost[i,i]=0.0 for $1 \leq i \leq n$

{

 for $i:=1$ to n do

```

        for j:=1 to n do
            A[i,j]:=cost[i,j]; //copy cost into A
    for k:=1 to n do
        for i:=1 to n do
            for j:=1 to n do
                A[i,j]:=min{A[i,j], A[i,k]+ A[k,j];
    }

```

COST ADJACENCY MATRIX :

- ❖ At 0th iteration it nil give you the direct distances between any 2 nodes

$$\text{cost}(i,j) = A(i,j) = D_0 = \begin{vmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{vmatrix}$$

It gives the direct distances between the two nodes.

- ❖ At 1st iteration we have to check the each pair(i,j) whether there is a path through node 1.if so we have to check whether it is minimum than the previous value and if i is so than the distance through 1 is the value of d1(i,j).at the same time we have to solve the intermediate node in the matrix position p(i,j).

```

for k= 1 to 4
When k=1
    for i= 1 to 4
when i=1
    for j= 1 to 4
        when j=1
            A[1,1]=min(A[1,1],a[1,1]+A[1,1])=min(0,0)=0
        When j=2
            A[1,2]=min(A[1,2],A[1,1]+A[1,2])=min(5,0+5)=5
        When j=3
            A[1,3]=min(A[1,3],A[1,1]+A[1,3])=min(∞,0+∞)=∞
        When j=4
            A[1,4]=min(A[1,4],A[1,1]+A[1,4])=min(∞,0+∞)=∞

    When i=2
        for j= 1 to 4
            when j=1
                A[2,1]=min(A[2,1],A[2,1]+A[1,1])=min(50,50+0)=50
            When j=2
                A[2,2]=min(A[2,2],A[2,1]+A[1,2])=min(0,50+5)=0
            When j=3
                A[2,3]=min(A[2,3],A[2,1]+A[1,3])=min(15,50+∞)=15
            When j=4
                A[2,4]=min(A[2,4],A[2,1]+A[1,4])=min(5,50+∞)=5

        When i=3
            for j= 1 to 4
                when j=1
                    A[3,1]=min(A[3,1],A[3,1]+A[1,1])=min(30,30+0)=30

```

When $j=2$

$$A[3,2] = \min(A[3,2], A[3,1] + A[1,2]) = \min(\infty, 30 + 5) = 35$$

When $j=3$

$$A[3,3] = \min(A[3,3], A[3,1] + A[1,3]) = \min(0, 30 + \infty) = 0$$

When $j=4$

$$A[3,4] = \min(A[3,4], A[3,1] + A[1,4]) = \min(15, 30 + \infty) = 15$$

When $i=4$

for $j=1$ to 4

when $j=1$

$$A[4,1] = \min(A[4,1], A[4,1] + A[1,1]) = \min(15, 15 + 0) = 15$$

When $j=2$

$$A[4,2] = \min(A[4,2], A[4,1] + A[1,2]) = \min(\infty, 15 + 5) = 20$$

When $j=3$

$$A[4,3] = \min(A[4,3], A[4,1] + A[1,3]) = \min(5, 15 + \infty) = 5$$

When $j=4$

$$A[4,4] = \min(A[4,4], A[4,1] + A[1,4]) = \min(0, 15 + \infty) = 0$$

k	i	j	A[i,j]
1	1	1	0
		2	5
		3	∞
		4	∞
	2	1	50
		2	0
		3	15
		4	5
	3	1	30
		2	35
		3	0
		4	15
	4	1	15
		2	20
		3	5
		4	0

$$D1 = \begin{vmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \mathbf{35} & 0 & 15 \\ 15 & \mathbf{20} & 5 & 0 \end{vmatrix} \quad \begin{matrix} P[3,2] = 1 \\ P[4,2] = 1 \end{matrix} \quad P_1 = \begin{vmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{vmatrix}$$

When $k=2$, we have obtained the matrix D_2 and P_2

$$D_2 = \begin{vmatrix} 0 & 5 & \mathbf{20} & \mathbf{10} \\ 50 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{vmatrix} \quad \begin{matrix} P[1,3] = 2 \\ P[1,4] = 2 \end{matrix} \quad P_2 = \begin{vmatrix} 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{vmatrix}$$

$$D_3 = \begin{vmatrix} 0 & 5 & 20 & 10 \\ \mathbf{45} & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{vmatrix} \quad \begin{matrix} P[2,1] = 3 \end{matrix}$$

$$D_4 = \begin{vmatrix} 0 & 5 & \mathbf{15} & 10 \\ 20 & 0 & \mathbf{10} & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{vmatrix} \quad \begin{matrix} P[1,3] = 4 \\ P[2,3] = 4 \end{matrix}$$

- D_4 will give the shortest distance between any pair of nodes.
- If you want the exact path then we have to refer the matrix p . The matrix will be,

$$P = \begin{vmatrix} 0 & 0 & 4 & 2 \\ 3 & 0 & 4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{vmatrix} \longrightarrow 0 \quad \text{direct path}$$

- Since, $p[1,3]=4$, the shortest path from 1 to 3 passes through 4.
- Looking now at $p[1,4]$ & $p[4,3]$ we discover that between 1 & 4, we have to go to node 2 but that from 4 to 3 we proceed directly.
- Finally we see the trips from 1 to 2, & from 2 to 4, are also direct.
- The shortest path from 1 to 3 is 1,2,4,3.

ALGORITHM :

Function Floyd ($L[1..r, 1..r]$): array $[1..n, 1..n]$
array $D[1..n, 1..n]$

$D = L$

For $k = 1$ to n do

For $i = 1$ to n do

For $j = 1$ to n do

$D[i, j] = \min(D[i, j], D[i, k] + D[k, j])$

Return D

ANALYSIS:

This algorithm takes a time of $\theta(n^3)$

4. Explain Travelling salesman problem (UQ APRIL'13 & NOV'12)

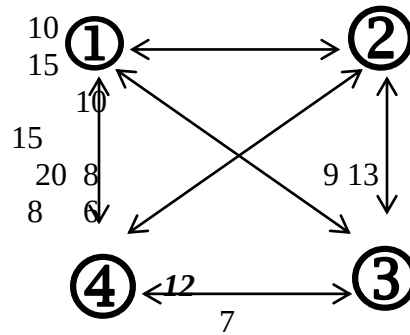
- Let $G(V, E)$ be a directed graph with edge cost c_{ij} is defined such that $c_{ij} > 0$ for all i and j and $c_{ij} = \infty$, if $\langle i, j \rangle \notin E$.
 - Let $|V| = n$ and assume $n > 1$.
- The traveling salesman problem is to find a tour of minimum cost.
- A tour of G is a directed cycle that include every vertex in V .
- The cost of the tour is the sum of cost of the edges on the tour.
- The tour is the shortest path that starts and ends at the same vertex (ie) 1.

Application:

1. Suppose we have to route a postal van to pick up mail from the mail boxes located at ' n ' different sites.
2. An $n+1$ vertex graph can be used to represent the situation.
3. One vertex represents the post office from which the postal van starts and return.
4. Edge $\langle i, j \rangle$ is assigned a cost equal to the distance from site ' i ' to site ' j '.
5. The route taken by the postal van is a tour and we are finding a tour of minimum length.
6. Every tour consists of an edge $\langle 1, k \rangle$ for some $k \in V - \{1\}$ and a path from vertex k to vertex 1.
7. The path from vertex k to vertex 1 goes through each vertex in $V - \{1, k\}$ exactly once.
8. The function which is used to find the path is
$$g(1, V - \{1\}) = \min\{c_{1j} + g(j, S - \{j\})\}$$
9. $g(i, S)$ be the length of a shortest path starting at vertex i , going through all vertices in S , and terminating at vertex 1.
10. The function $g(1, V - \{1\})$ is the length of an optimal tour.

STEPS TO FIND THE PATH:

1. Find $g(i, \Phi) = c_{i1}$, $1 \leq i \leq n$, hence we can use equation(2) to obtain $g(i, s)$ for all s to size 1.
2. That we have to start with $s=1$, (ie) there will be only one vertex in set 's'.
3. Then $s=2$, and we have to proceed until $|s| < n-1$.
4. for example consider the graph.



Cost matrix

	0	10	15	20
5	0	9	10	
6	13	0	12	
8	8	9	0	

$g(i, s) \longrightarrow$ set of nodes/vertex have to visited.

$\downarrow$
Starting position

$$g(i, s) = \min\{c_{ij} + g(j, s - \{j\})\}$$

STEP 1:

$$g(1, \{2, 3, 4\}) = \min\{c_{12} + g(2, \{3, 4\}), c_{13} + g(3, \{2, 4\}), c_{14} + g(4, \{2, 3\})\}$$

$$\min\{10 + 25, 15 + 25, 20 + 23\}$$

$$\min\{35, 35, 43\}$$

$$= 35$$

STEP 2:

$$g(2, \{3, 4\}) = \min\{c_{23} + g(3, \{4\}), c_{24} + g(4, \{3\})\}$$

$$\min\{9 + 20, 10 + 15\}$$

$$\min\{29, 25\}$$

$$= 25$$

$$g(3,\{2,4\}) = \min\{c_{32}+g(2\{4\}), c_{34}+g(4,\{2\})\}$$

$$\min\{13+18, 12+13\}$$

$$\min\{31, 25\}$$

$$=25$$

$$g(4,\{2,3\}) = \min\{c_{42}+g(2\{3\}), c_{43}+g(3,\{2\})\}$$

$$\min\{8+15, 9+18\}$$

$$\min\{23, 27\}$$

$$=23$$

STEP 3:

$$1. \quad g(3,\{4\}) = \min\{c_{34} + g\{4,\Phi\}\}$$

$$12+8 = 20$$

$$2. \quad g(4,\{3\}) = \min\{c_{43} + g\{3,\Phi\}\}$$

$$9+6 = 15$$

$$3. \quad g(2,\{4\}) = \min\{c_{24} + g\{4,\Phi\}\}$$

$$10+8 = 18$$

$$4. \quad g(4,\{2\}) = \min\{c_{42} + g\{2,\Phi\}\}$$

$$8+5 = 13$$

$$5. \quad g(2,\{3\}) = \min\{c_{23} + g\{3,\Phi\}\}$$

$$9+6 = 15$$

$$6. \quad g(3,\{2\}) = \min\{c_{32} + g\{2,\Phi\}\}$$

$$13+5 = 18$$

STEP 4:

$$g\{4,\Phi\} = c_{41} = 8$$

$$g\{3,\Phi\} = c_{31} = 6$$

$$g\{2,\Phi\} = c_{21} = 5$$

$$\left| s \right| = 0.$$

i =1 to n.

$$g(1,\Phi) = c_{11} \Rightarrow 0$$

$$g(2,\Phi) = c_{21} \Rightarrow 5$$

$$g(3,\Phi) = c_{31} \Rightarrow 6$$

$$g(4,\Phi) = c_{41} \Rightarrow 8$$

$$\left| s \right| = 1$$

i =2 to 4

$$g(2,\{3\}) = c_{23} + g(3,\Phi)$$

$$= 9+6 =15$$

$$g(2,\{4\}) = c_{24} + g(4,\Phi)$$

$$= 10+8 =18$$

$$g(3,\{2\}) = c_{32} + g(2,\Phi)$$

$$= 13+5 =18$$

$$g(3,\{4\}) = c_{34} + g(4,\Phi)$$

$$= 12+8 =20$$

$$g(4,\{2\}) = c_{42} + g(2,\Phi)$$

$$= 8+5 =13$$

$$g(4,\{3\}) = c_{43} + g(3,\Phi)$$

$$= 9+6 =15$$

$$s = \left| 2 \right|$$

i ≠ 1, 1 ∈ s and i ∈ s.

$$\begin{aligned}
 g(2,\{3,4\}) &= \min\{c_{23}+g(3\{4\}),c_{24}+g(4,\{3\})\} \\
 &= \min\{9+20,10+15\} \\
 &= \min\{29,25\} \\
 &= 25
 \end{aligned}$$

$$\begin{aligned}
 g(3,\{2,4\}) &= \min\{c_{32}+g(2\{4\}),c_{34}+g(4,\{2\})\} \\
 &= \min\{13+18,12+13\} \\
 &= \min\{31,25\} \\
 &= 25
 \end{aligned}$$

$$\begin{aligned}
 g(4,\{2,3\}) &= \min\{c_{42}+g(2\{3\}),c_{43}+g(3,\{2\})\} \\
 &= \min\{8+15,9+18\} \\
 &= \min\{23,27\} \\
 &= 23
 \end{aligned}$$

$$|s| = 3$$

$$\begin{aligned}
 g(1,\{2,3,4\}) &= \min\{c_{12}+g(2\{3,4\}),c_{13}+g(3,\{2,4\}),c_{14}+g(4,\{2,3\})\} \\
 &= \min\{10+25,15+25,20+23\} \\
 &= \min\{35,35,43\} \\
 &= 35
 \end{aligned}$$

optimal cost is 35

the shortest path is,

$$g(1,\{2,3,4\}) = c_{12} + g(2,\{3,4\}) \Rightarrow 1 \rightarrow 2$$

$$g(2,\{3,4\}) = c_{24} + g(4,\{3\}) \Rightarrow 1 \rightarrow 2 \rightarrow 4$$

$$g(4,\{3\}) = c_{43} + g(3\{\Phi\}) \Rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$$

So the optimal tour is **1 → 2 → 4 → 3 → 1**

5. Explain chained matrix multiplication (UQ NOV'12 & NOV'10)

If we have a matrix A of size $p \times q$ and B matrix of size $q \times r$. the product of these two matrixes C is given by,

$$c_{ij} = \sum a_{ik} b_{kj}, 1 \leq i \leq p, 1 \leq j \leq r, 1 \leq k \leq q.$$

it requires a total of pqr scalar multiplication.

Matrix multiplication is associative, so if we want to calculate the product of more than 2 matrixes $m_1 m_2 \dots m_n$.

For example,

$$A = 13 \times 5$$

$$B = 5 \times 89$$

$$C = 89 \times 3$$

$$D = 3 \times 34$$

Now, we will see some of the sequence,

$$\begin{aligned} M &= (((A.B).C).D) \\ &\quad A.B \quad C \\ &= (13 * 5 * 89) * (89 * 3) \\ &\quad A.B.C. \quad D \\ &= (13 * 89 * 3) * (3 * 34) \\ &\quad A.B.C.D \\ &= 13 * 3 * 34 \end{aligned}$$

$$\begin{aligned} (ic) &= 13 * 5 * 89 + 13 * 89 * 3 + 13 * 3 * 34 \\ &= 10,582 \text{ no. of multiplications one required for that sequence.} \end{aligned}$$

2nd Sequence,

$$\begin{aligned} M &= (A * B) * (C * D) \\ &= 13 * 5 * 89 + 89 * 3 * 34 + 13 * 89 * 34 \\ &= 54201 \text{ no. of Multiplication} \end{aligned}$$

3rd Sequence,

$$\begin{aligned} M &= (A.(BC)) . D \\ &= 5 * 89 * 3 + 13 * 5 * 3 + 13 * 3 * 34 \\ &= 2856 \end{aligned}$$

For comparing all these sequence, $(A(BC)).D$ sequences less no. of multiplication.

For finding the no. of multiplication directly we are going to the Dynamic programming method.

STRAIGHT FORWARD METHOD:

Our aim is to find the total no. of scalar multiplication required to compute the matrix product.

Let $(M_1, M_2, \dots, M_i)$ and $M_{i+1}, M_{i+2}, \dots, M_n$ be the chain of matrix to be calculated using the dynamic programming method.

In dynamic programming we always start with the smallest instances and continue till we reach the required size.

We maintain a table m_{ij} , $1 \leq i \leq j \leq n$,
Where m_{ij} gives the optimal solution.

Sizes of all the matrixes are stored in the array $d[0..n]$

We build the table diagonal by diagonal; diagonal s contains the elements m_{ij} such that $j-i = s$.

RULES TO FILL THE TABLE M_{ij} :

$S = 0, 1, \dots, n-1$

If $s=0 \Rightarrow m(i,i) = 0, i = 1, 2, \dots, n$

If $s=1 \Rightarrow m(i, i+1) = d(i-1) * d_i * d(i+1)$

$i = 1, 2, \dots, n-1.$

If $1 < s < n \Rightarrow m_{i, i+s} = \min(m_{ik} + m_{k+1, i+s} + d_{i-1} d_k d_{i+s})$
 $i \leq k \leq i+s \quad i = 1, 2, \dots, n-s$

Apply this to the example,

$A \Rightarrow 13 \times 5$

$B \Rightarrow 5 \times 89$

$C \Rightarrow 89 \times 3$

$D \Rightarrow 3 \times 34$

Single dimension array is used to store the sizes.

$d[0] = 13$

$d[1] = 5$

$d[2] = 89$

$d[3] = 3$

$d[4] = 34$

if $s=0$,

$m(1,1) = 0$

$m(2,2) = 0$

$m(3,3) = 0$

$m(4,4) = 0$

if $s=1$,

$$m_{i,i+1} = d(i-1) * d_i * d(i+1)$$

$$m_{12} = d_0 * d_1 * d_2 = 13 * 5 * 89 = 5785$$

$$m_{23} = d_1 * d_2 * d_3 = 5 * 89 * 3 = 1335$$

$$m_{34} = d_2 * d_3 * d_4 = 89 * 3 * 34 = 9078$$

if $s=2$,

$$m_{i,i+s} = \min(m_{ik} + m_{k+1,i+s} + d_{i-1} d_k d_{i+s})$$

$$m_{13} = \min(m_{11} + m_{23} + d_0 d_1 d_3, m_{12} + m_{33} + d_0 d_2 d_3)$$

$$= (0 + 1335 + (13 * 5 * 3), 5785 + 0 + (13 * 89 * 3))$$

$$= \min(1530, 9256)$$

$$= 1530$$

$$m_{13} = \min(m_{22} + m_{34} + d_1 d_2 d_4, m_{23} + m_{44} + d_1 d_3 d_4)$$

$$= (0 + 9078 + (5 * 89 * 34), 1335 + 0 + (5 * 3 * 34))$$

$$= \min(24208, 1845)$$

$$= 1845$$

$$m_{14} = \min(m_{11} + m_{24} + d_0 d_1 d_4, m_{12} + m_{34} + d_0 d_2 d_4, m_{13} + m_{44} + d_0 d_3 d_4)$$

$$= \min(4055, 54201, 2856)$$

$$= 2856$$

the matrix table m_{ij} will look like,

	1	2	3	4	
1	0	5785	1530	2856	$s=3$
2		0	1335	1845	$s=2$
3			0	9078	$s=1$
4				0	$s=0$

Here there is no need to fill the lower diagonal.

ALGORITHM:

Procedure cmatrix(n,d[0..n])

```
For s=0 to n-1 do
{
    if(s==0)
        for I=1 to n do
            m(i,j) =0

    if(s==1)
        for i=1 to n-1 do
            m(i,i+1) =d(i-1)*d(i)*d(i+1)

    else
    {
        m=∞
        for i=1 to n-s do
            for k=i to i+s do
            {

                if (min>[m(i,k) +m(k+1,i+s)+d(i-1)*d(k)*d(i+s)])

                    min=[m(i,k) +m(k+1,i+s)+d(i-1)*d(k)*d(i+s)]

            }

            m(i,i+s) =min

        }

    }
}
```

6. Explain 0/1 knapsack problem (UQ APRIL'12)

- This problem is similar to ordinary knapsack problem but we may not take a fraction of an object.
- We are given ' N ' object with weight W_i and profits P_i where i varies from 1 to N and also a knapsack with capacity ' M '.
- The problem is, we have to fill the bag with the help of ' N ' objects and the resulting profit has to be maximum.
- Formally, the problem can be stated as, maximize $\sum_{i=1}^n X_i P_i$

$$i=1$$

$$\text{subject to } \sum_{i=1}^n X_i W_i \leq M$$

- Where X_i are constraints on the solution $X_i \in \{0,1\}$. (u) X_i is required to be 0 or 1. if the object is selected then the unit is 1. If the object is rejected then the unit is 0. That's why it is called as 0/1, knapsack problem.
- To solve the problem by dynamic programming we use a table $T[1 \dots N, 0 \dots M]$ (ic) the size is $N \times M$. where 'N' is the no. of objects and column starts with '0' to capacity (ic) 'M'.
- In the table $T[i,j]$ will be the maximum value of the objects i varies from 1 to n and j varies from 0 to M .

RULES TO FILL THE TABLE:-

- If $i=1$ and $j < w(i)$ then $T(i,j) = 0$, (ic) 0 is filled in the table.
- If $i=1$ and $j \geq w(i)$ then $T(i,j) = p(i)$, the cell is filled with the profit $p[i]$, since only one object can be selected to the maximum.
- If $i>1$ and $j < w(i)$ then $T(i,j) = T(i-1,j)$ the cell is filled the profit of previous object since it is not possible with the current object.
- If $i>1$ and $j \geq w(i)$ then $T(i,j) = \max\{p(i) + T(i-1, j-w(i)), T(i-1, j)\}$, since only '1' unit can be selected to the maximum. If is the current profit + profit of the previous object to fill the remaining capacity of the bag.
- After the table is generated it will give details the profit.

ES TO GET THE COMBINATION OF OBJECT:

- Start with the last position of i and j , $T[i,j]$, if $T[i,j] = T[i-1,j]$ then no object of 'i' is required so move up to $T[i-1,j]$.
- After moved, we have to check if, $T[i,j] = T[i-1, j-w(i)] + p[i]$, if it is equal then one unit of object 'i' is selected and move up to the position $T[i-1, j-w(i)]$
- Repeat the same process until we reach $T[i,0]$, then there will be nothing to fill the bag stop the process.
- Time is $O(nw)$ is necessary to construct the table T .
- Consider a Example,

$$\begin{aligned} M &= 6, \\ N &= 3 \\ W_1 &= 2, W_2 = 3, W_3 = 4 \\ P_1 &= 1, P_2 = 2, P_3 = 5 \end{aligned}$$

i $\longrightarrow$ 1 to N
j $\longrightarrow$ 0 to 6

i=1, j=0 (ic) i=1 & j < w(i)

0 < 2 $T_{1,0} = 0$

i=1, j=1 (ic) i=1 & j < w(i)

1 < 2 $T_{1,1} = 0$ (Here j is equal to w(i)) P(i) $\longrightarrow$

i=1, j=2
2 > 2, $T_{1,2} = 1$.

i=1, j=3
3 > 2, $T_{1,3} = 1$.

i=1, j=4
4 > 2, $T_{1,4} = 1$.

i=1, j=5
5 > 2, $T_{1,5} = 1$.

i=1, j=6
6 > 2, $T_{1,6} = 1$.

=> i=2, j=0 (ic) i > 1, j < w(i)
0 < 3 = $T(2,0) = T(i-1,j) = T(1,0)$
 $T_{2,0} = 0$

i=2, j=1
1 < 3 = $T(2,1) = T(i-1,j)$
 $T_{2,1} = 0$

7. Explain approaches using recursion

Although dynamic programming algorithms give efficient result, there is some unsatisfactory about the bottom up approach. Because it leads to compute values that might be completely irrelevant. So we go for the top down approach to achieve the same efficiency.

The matrix multiplication using dynamic programming can be replaced that means replacing the table M by a function fm which is calculated as required in other words we would like to find a function fm such that fm(I,j) equal to mi,j for $1 \leq I \leq j \leq n$; But that can be calculated recursively, writing such a function is simple. We have to program the rules for calculating M.

Function fm(i,j)

```
if i=j then {only one matrix is involved}
return 0;
m=infinity
for k= I to j-1 do
m=min
```

MEMORY FUNCTIONS:

Procedure cmatrix(n,d[0..n])

```
For s=0 to n-1 do
{
    if(s==0)

    for I=1 to n do

        m(i,j) =0

    if(s==1)
    for i=1 to n-1 do
        m(i,i+1) =d(i-1)*d(i)*d(i+1)

    else
    {
        m= $\infty$ 
        for i=1 to n-s do
            for k=i to i+s do

            {

                if (min>[m(i,k) +m(k+1,i+s)+d(i-1)*d(k)*d(i+s)])

                    min=[m(i,k) +m(k+1,i+s)+d(i-1)*d(k)*d(i+s)]

            }

            m(i,i+s) =min

        }

    }
}
```

8. Explain basic search and traversal technique (UQ APRIL'13/ APRIL 2015)

TECHNIQUES FOR GRAPHS:

- The fundamental problem concerning graphs is the reach-ability problem.
- In its simplest form it requires us to determine whether there exists a path in the given graph, $G=(V,E)$ such that this path starts at vertex 'v' and ends at vertex 'u'.
- A more general form is to determine for a given starting vertex $v \in V$ all vertex 'u' such that there is a path from v to u.

- This problem can be solved by starting at vertex 'v' and systematically searching the graph 'G' for vertex that can be reached from 'v'.
- We describe 2 search methods for this.
 - i. Breadth first Search and Traversal.
 - ii. Depth first Search and Traversal.

Breadth first search and traversal:

- In Breadth First Search we start at a vertex 'v' and mark it as having been reached (visited).
- The vertex 'v' is at this time said to be unexplored.
- A vertex is said to have been explored by an algorithm when the algorithm has visited all vertices adjacent from it.
- All unvisited vertices adjacent from 'v' are visited next. These are new unexplored vertices.
- Vertex 'v' has now been explored. The newly visit vertices haven't been explored and are put on the end of a list of unexplored vertices.
- The first vertex on this list is the next to be explored. Exploration continues until no unexplored vertex is left.
- The list of unexplored vertices operates as a queue and can be represented using any of the standard queue representation.

ALGORITHM:

Algorithm BFS (v)

```
// A breadth first search of 'G' is carried out.
// beginning at vertex-v; For any node i, visit.
// if 'i' has already been visited. The graph 'v'
// and array visited [] are global; visited []
// initialized to zero.
{
  y=v; // q is a queue of unexplored 1visited (v)= 1
  repeat
  {
    for all vertices 'w' adjacent from u do
    {
      if (visited[w]=0) then
      {
        Add w to q;
        visited[w]=1
      }
    }
    if q is empty then return; // No delete u from q;
```

```

} until (false)
}

```

algorithm : breadth first traversal

```

algorithm BFT(G,n)

{
    for i= 1 to n do
        visited[i] =0;
    for i =1 to n do
        if (visited[i]=0)then BFS(i)
}

```

here the time and space required by BFT on an n -vertex e -edge graph one $O(n+e)$ and $O(n)$ respectively if adjacency list is used .if adjacent matrix is used then the bounds are $O(n^2)$ and $O(n)$ respectively

Depth first search and traversal

A depth first search of a graph differs from a breadth first search in that the exploration of a vertex v is suspended as soon as a new vertex is reached. At this time the exploration of the new vertex u begins. When this new vertex has been explored, the exploration of u continues. The search terminates when all reached vertices have been fully explored. This search process is best-described recursively.

```

ALGORITHM DFS(V)
//given an undirected (directed graph)
//G=(V,E) with n vertices and an array visited[]
//initially set to zero, this algorithm of visits all the vertices reachable
//from v.
//G and visited[] are global
{
    visited[v]:=1;
    for each vertex w adjacent from v do
        {
            if(visited[w]=0) then DFS(w);
        }
}

```

9. Explain biconnected components (UQ:APRIL/MAY'14)

Articulation Point:

A vertex ' v ' in a connected graph ' G ' is an articulation point, iff the deletion of vertex v together with all vertices incident to v disconnects the graph into 2 or more non empty components.

Biconnected Graph:

A graph ' G ' is biconnected, iff it contains no articulation points.

Real Time Example:

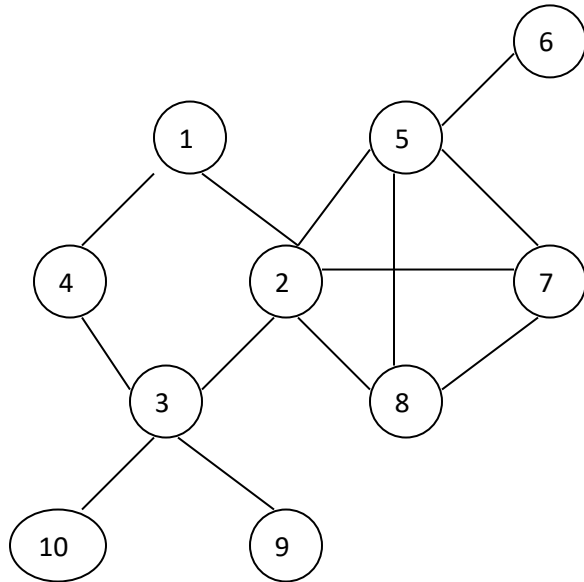
' G ' is a graph representing a communication network, vertices represent communication station, edges represent communication lines.

(i) if communication station has articulation point :

→ if any communication station 'i' is failed , then we cannot communicate with another station.

(ii) if the graph has no articulation point:

→if any station 'i' is failed, then we can communicate with another station, other than 'i'.

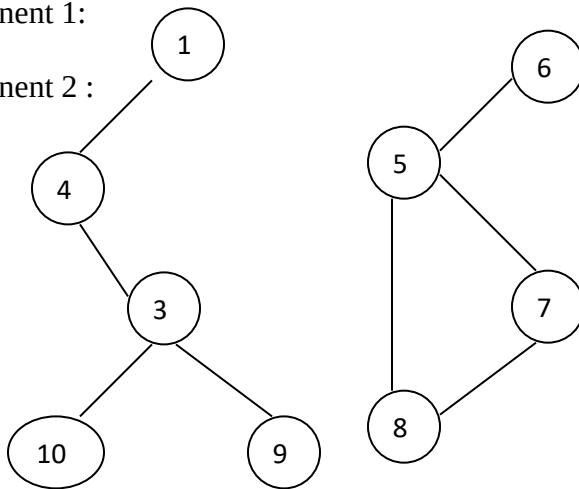


Figure– Graph G

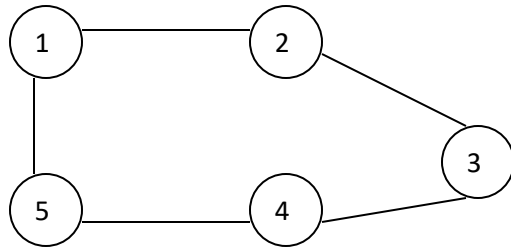
In the above graph G ,if we delete the vertex 2 ,then it creates 2 non empty components.

Component 1:

Component 2 :



If we delete any other vertex other than 2, 3, and 5, then we get only one component exactly. So the above graph is not biconnected.



The figure represents the graph which is biconnected. Because it has no articulation points.

If a graph is not biconnected, then we may include set of edges that makes the graph biconnected .we can determine that edge if we know the maximal subgraphs of G that are biconnected.

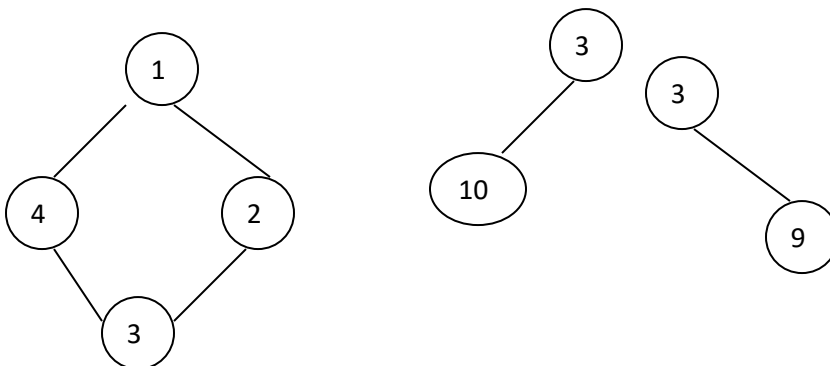
Let $G(V,E)$ be a graph that is not biconnected. To make it biconnected , first identify the articulation points by the algorithm. Once it has been determined that a connected graph G is not biconnected, it may be desirable to determine the set of edges whose inclusion makes the graph biconnected. Determining such a set of edges is facilitated if we know the maximal subgraphs of G that are biconnected. $G'(V',E')$ is a maximal biconnected subgraph of G iff G has no biconnected subgraph $G''(V'',E'')$ such that $V' \subset V''$ and $E' \subset E''$. A maximal biconnected subgraph is a biconnected component.

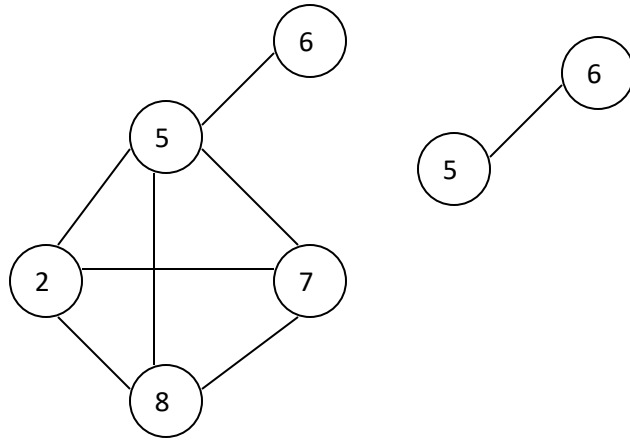
Biconnected components should contain atleast 2 vertices. Biconnected component itself has no articulation point. The graph G can be transformed into a biconnected graph by using the edge addition scheme of Algorithm to construct a biconnected graph. Note that once the edges (v_i, v_{i+1}) of line 6 are added, vertex a is no longer an articulation point. Hence following the addition of edges corresponding to all articulation points, G has no articulation points and so it is biconnected. If G has p articulation points and b biconnected components, the scheme of algorithm introduces exactly $b-p$ new edges into G .

Algorithm To Construct A Biconnected Graph:

```

1  for each articulation point 'a' do
2  {
3      let  $B_1, B_2, \dots, B_k$  be the biconnected components containing vertex  $a$ ;
4      let  $V_i, V_{i+1} \neq a$  be a vertex in  $B_i$ ,  $1 \leq i \leq k$ ;
5      add to ' $G$ ' the edges  $(V_i, V_{i+1}), 1 \leq i \leq k$ ;
6  }
```





Pseudo Code To Determine Bicomponents :

Algorithm Bicomp (u,v)

```

{
    //u is the start vertex for depth first search.
    // v is its parents ,if any in the depth first spanning tree .
    //it is assumed that ,the global array 'dfn' is initially zero and that the global variable
    // 'num' is initialized to 1.
    //'n' is the number of vertices
    {
        dfn[u]:=num;L[u]:=num;num=num+1;
        for each vertex 'a' adjacent from 'u' do
            {
                if((v!=w)&&(dfn[w]<dfn[u])) then
                    add [u,w] to the top of the stack S;
                if(dfn[w]=0) then
                    {
                        if(L[w]>=dfn[u]) then
                            {
                                write("New Bicomponents");
                                repeat
                                    {
                                        delete an edge from top of stack s;
                                        let the edge be (x,y);
                                        write(x,y);
                                    }
                                until(((x,y)=(u,w))||((x,y)=(w,u)));
                            }
                        Bicomp(w,u); //w is unvisited
                        L(u):=min(L(u),L(w));
                    }
                else if(w!=v) then
                    L(u):=min(L(u),dfn[w]);
            }
        }
    }
}

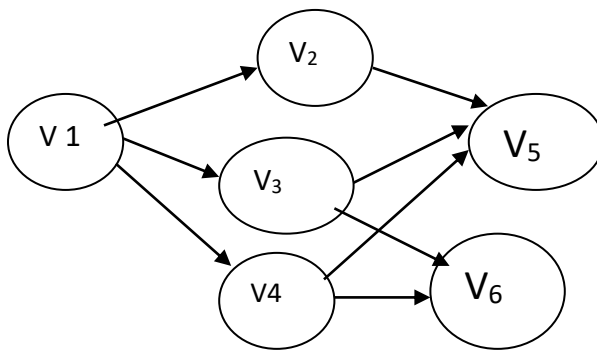
```

10. Explain topological sorting (UQ NOV'10, APRIL/MAY'14)

A directed graph G in which the vertices represent tasks/events and the edges represent activities to move from one event to another then that graph is known as activity on vertex network/AOV-network/PERT graph.

PERT graph is made to analyze interrelated activities of complete projects. The purpose of a topological sort is to order the events in a linear manner, with the restriction that an event cannot precede other events that must first take place.

A topological sort defines a linear ordering on these orders of a directed graph that have the following property: If node P is a predecessor of node Q , then Q cannot be the predecessor of node P . A topological sort cannot order the nodes in a cycle.



$V_1, V_2, V_5, V_3, V_6, V_4$

$V_1, V_4, V_3, V_6, V_2, V_5$

Fig(b) Topological order .

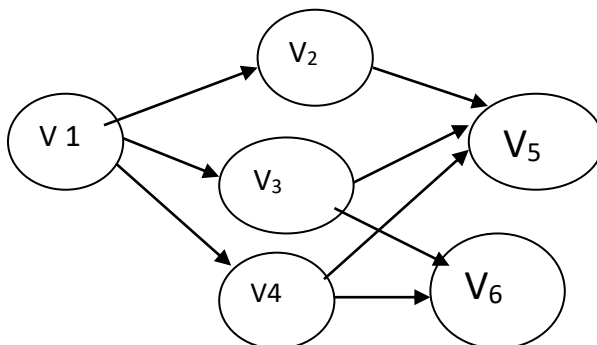
ALGORITHM: To sort the tasks into topological order:

Let n be the no of vertices

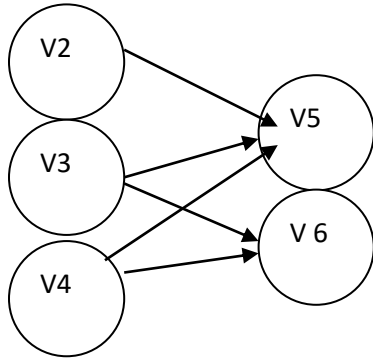
1. For $i \rightarrow 1$ to n do //o/p the vertices//
2. If every vertex has no predecessor then (the network has a cycle and stop)
3. Pick a vertex i.e. which has no predecessor.
4. O/p V
5. Delete V and all edges leading and of V from the network.
6. End.

Apply this for the above given graph, we will get:

a) Initial



b) V_1



Topological order generated is:

$V_1, V_4, V_3, V_6, V_2, V_5$.

Here on step (b) we have, 3 vertices V_2, V_3 & V_4 which has no predecessor and any of these can be the next vertex in topological order.

In this problem, the functions are:

- i) Decide whether a vertex has any predecessor.
- ii) Decide a vertex together with all its incident edges.

- This can be efficiently done if for each vertex a count of the no of its immediate predecessor is kept.
- This can be easily implemented if the network is represented by its adjacency lists.

The following algorithm assumes that the network is represented by adjacency lists.

ALGORITHM:

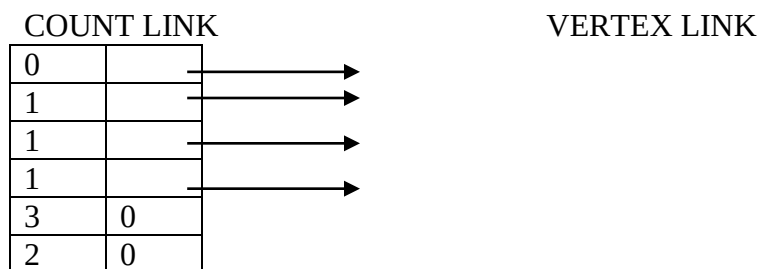
Procedure Topological_order (count,vertex,link,n)

//the n vertices of an AOV-network are listed in topological order .

the network is represented as a set of adjacency lists with

COUNT(i)=the in-degree of vertex i//

- 1) $top \leftarrow 0$ //initialize stack//
- 2) for $i \leftarrow 1$ to n do //create a linked stack of vertices with no predecessors //
- 3) if COUNT (I)=0 then [COUNT(i) $\leftarrow$ top;top $\leftarrow$ I]
- 4) end
- 5) for $i \leftarrow 1$ to n do //print the vertices in topological order //
- 6) if top = 0 then [print ('network has a cycle ');stop]
- 7) $j \leftarrow$ top;top $\leftarrow$ COUNT (top);print(j)//unstack a vertex//
- 8) $ptr \leftarrow$ LINK(j)
- 9) while ptr $\neq$ 0 do
- 10) //decrease the count of successor vertices of j//
- 11) $K \leftarrow$ VERTEX(ptr) // K is a successor of j//
- 12) COUNT(K) $\leftarrow$ COUNT(K)-1 // Decrease count//
- 13) If COUNT(K)=0 // Add vertex K to stack//
- 14) Then[COUNT(K) $\leftarrow$ top; top $\leftarrow$ K]
- 15) $Ptr \leftarrow$ LINK(ptr)
- 16) End
- 17) End
- 18) End TOPOLOGICAL_ORDER.



The head nodes of these lists contain two fields: COUNT & LINK. The COUNT field contains the in-degree of that vertex and LINK is a pointer to the first node on the adjacency list. Each list node has 2 fields: VERTEX & LINK.

COUNT Fields can be set at the time of i/p. When edge $\langle i, j \rangle$ is i/p the count of vertex j is incremented by 1. The list of vertices with zero count is maintained as a stack.

Pondicherry University Questions

2 Marks

- 1 Define dynamic programming. (UQ APRIL'12) (Ref.Pg.No.1 Qn.No.2)
2. Write the general procedure of dynamic programming. (UQ APRIL'13) (Ref.Pg.No.1 Qn.No.5)
3. List different traversal technique in graph (UQ APRIL'12) (Ref.Pg.No.77 Qn.No.2)
4. What is biconnected component? (UQ Nov'12) (Ref.Pg.No.2 Qn.No.15)
5. Write about multistage graph with example (UQ APRIL'13 & Nov'12,Apr/May'14) (Ref.Pg.No.2 Qn.No.17)
6. What is all pair shortest path problem?(APRIL/MAY'14/May'16) (Ref.Pg.No.2 Qn.No.18)

11 Marks

1. Explain Travelling salesman problem (UQ APRIL'13 & NOV'12) (Ref.Pg.No.13 Qn.No.4)
2. Explain chained matrix multiplication (UQ NOV'12 & NOV'10/May'16) (Ref.Pg.No.18 Qn.No.5)
3. Explain 0/1 knapsack problem (UQ APRIL'12) (Ref.Pg.No.21 Qn.No.6)
4. Explain basic search and traversal technique (UQ APRIL'13) (Ref.Pg.No.24 Qn.No.8)
5. Explain topological sorting (UQ NOV'10, APRIL/MAY'14) (Ref.Pg.No. 30 Qn.No.10)
6. Explain multistage graph with example (UQ :APRIL/MAY'14/Nov'16) (Ref.Pg.No.3,Qn.No.2)
- 7.Explain Biconnected components (UQ:APRIL/MAY'14)) (Ref.Pg.No.26,Qn.No.9)
- 8.What are AND/OR graph?Explain in detail.(Nov'16)